The science of pretraining: annotated deep-dive notes

This Quarto edition is generated from the preserved `scaling_ladder_notes.md`. The notes' prose and equations are retained; the original top-level title is supplied through YAML, and companion-reading links are editorial additions. Regenerate with `python scripts/build_notes_quarto.py`.

Important

Read the primary-source review alongside these notes. In particular, the constant AdamW-timescale statement needs a cross-TPP qualification. Numerical claims in this synthesis are not all independently audited.

Start with the blog reading guide for the section crosswalk, or use the source housekeeping page to locate papers and code.

For dedicated paper methods, read Step Law, Farseer, and Power Lines. Marin replication is covered in the optimizer search and Datakit code manual.

For the complete efforts, read OLMo, Marin, and the beginning-to-end pretraining design.

Technical notes on Jiaxuan Zou, "How to Build a Scientific Scaling Ladder" (Sept 2026), restructured, with derivations added.

> **Scope note.** The post is a synthesis of public papers and tech reports. Several cited works are 2026 reports (e.g., Qwen3.8-Next, Kimi K3, Skaling, Delphi) whose numbers I report as the post states them. Where I add derivations or flag a likely inaccuracy, it is marked **[Note]**.

---

0. Executive summary

Companion reading

A **Scaling Ladder** is a designed grid of small training runs over parameter count N, token count D (and other controlled axes) used to fit empirical laws, then extrapolate to a flagship run: the optimal (N, D), hyperparameters (η, B, λ), the final loss, and downstream scores.

Why pretraining *looks* predictable: when the recipe is held fixed and every small point is properly tuned, held-out cross-entropy is a smooth, nearly power-law function of (N, D) over many orders of magnitude. Why it is *not automatically* predictable:

1. **Competitiveness and predictability are different properties.** A recipe can reach low loss at small scale yet extrapolate badly (Marin's Delphi: first recipe was 2.5% above prediction at 10^{22} FLOPs and diverged at 10^{23}).
2. **Under-tuned small models bias the exponent.** Clean laws only appear on the *fully-tuned frontier*.
3. **Functional-form misspecification** (e.g., the additive Chinchilla form) creates structured residuals that blow up under extrapolation.
4. **Stability failures** (loss spikes, overflow, router collapse) are often dormant at small scale.
5. **Downstream accuracy is a discontinuous transform of loss**, so it is much less predictable than loss.
6. **Implementation drift**: the small-scale and production stacks may not compute the same update.

The rest of these notes are the engineering protocol for controlling each of these.

---

1. Why pretraining is predictable: the mathematical core

ⓘ Companion reading

1.1 The basic law and compute-optimal allocation

Read the detailed companion →

The Chinchilla parametric form is

$$L(N, D) = E + \frac{A}{N^\alpha} + \frac{B}{D^\beta},$$

with E the irreducible entropy of the text distribution. With the approximation $C = 6ND$, minimize L subject to fixed C. Substituting $D = C/(6N)$ and setting $\partial L/\partial N = 0$:

$$-\alpha A N^{-\alpha-1} + \beta B \left(\tfrac{C}{6}\right)^{-\beta} N^{\beta-1} = 0 \implies N_{\text{opt}}(C) = G\left(\frac{C}{6}\right)^{\frac{\beta}{\alpha+\beta}}, \quad D_{\text{opt}}(C) = G^{-1}\left(\frac{C}{6}\right)^{\frac{\alpha}{\alpha+\beta}},$$

where $G = \left(\frac{\alpha A}{\beta B}\right)^{\frac{1}{\alpha+\beta}}$. Chinchilla's empirical finding $a \approx b \approx 0.5$ in $N_{\text{opt}} \propto C^a, D_{\text{opt}} \propto C^b$ corresponds to $\alpha \approx \beta$. The famous $D/N \approx 20$ counts *total* parameters including embeddings; with trunk-only N_{body}, the optimal TPP is higher and is corpus/recipe dependent.

1.2 The coupled ("Skaling") form

Read the detailed companion →

The additive law forces $\partial^2 L/\partial N \partial D \equiv 0$ (no interaction). Skaling (Videau et al., 2026) uses

$$L(N, D) = \left(\frac{A}{N^\alpha} + \frac{B}{D^\beta}\right)^k + E, \qquad k = 1 \Rightarrow \text{Chinchilla; fitted } k \approx 0.31\text{–}0.45.$$

[Note] Two properties, derived. Let $f = AN^{-\alpha} + BD^{-\beta}$.

Interaction sign.

$$\frac{\partial^2 L}{\partial N\, \partial D} = k(k-1)\, f^{k-2}\, \alpha\beta AB\, N^{-\alpha-1} D^{-\beta-1}.$$

For $0 < k < 1$ this is strictly negative: increasing D makes $\partial L/\partial N$ more negative, i.e., N and D are complements.

Same compute-optimal allocation. Since $x \mapsto x^k + E$ is strictly increasing on $x > 0$, $\arg\min_{N:\, 6ND=C} L = \arg\min_{N:\, 6ND=C} f$, so $(N_{\text{opt}}, D_{\text{opt}})$ is exactly the Chinchilla closed form of §1.1. The extra parameter changes the *loss surface* (and hence the predicted loss at the optimum) but not the allocation formula.

Empirically, Chinchilla's residuals on a dense (N, D) grid are saddle-shaped, growing toward the four corners (very under- and over-trained). Skaling's residuals are near zero across the grid; it is more accurate on 76% of configurations with a median error ratio of $2.2\times$, and with 6 parameters it beats the 9-parameter Farseer form on interpolation and single-axis extrapolation.

Cheaper grids. With Skaling, an **L-shape** layout (vary N only at small D, vary D only at small N) costs about $1/5$–$1/10$ of a full grid and still extrapolates into the large-(N, D) interior; Chinchilla drifts badly from an L-shape.

Fixed-ratio degeneracy. If every run uses the same $M = D/N$, the surface collapses to $L = G(M)\,C^{-\gamma} + E$ and N, D exponents are not separately identifiable (see §4.1).

1.3 Equivalent compute multiplier (the right unit for "how big is a gain?")

Read the detailed companion →

For a one-dimensional law $L(C) = E + AC^{-\gamma}$, two recipes reaching losses L_1 and $L_2 = L_1 - \Delta L$ at the same compute differ by an equivalent compute factor

$$\frac{C_2}{C_1} = \left(\frac{L_1 - E}{L_1 - E - \Delta L}\right)^{1/\gamma} \quad \Longrightarrow \quad \ln\frac{C_2}{C_1} = \frac{1}{\gamma}\ln\left(1 - \frac{\Delta L}{L_1 - E}\right)^{-1} \approx \frac{\Delta L}{\gamma\,(L_1 - E)}$$

to first order in $\Delta L/(L_1 - E)$. Because $L - E \to 0$ with scale, a fixed ΔL is worth an increasingly large compute multiplier. Hence acceptance bars should be stated in equivalent compute (e.g., "15% compute savings") *and* absolute ΔL, not relative loss error.

1.4 Where predictability breaks (summary of failure modes)

Read the detailed companion →

Failure mode	Mechanism	Mitigation (section)
Under-tuned small points	Small models are sharply sensitive to HPs; inflated losses tilt the slope	Tune onto fully-tuned frontier (§5.1)
Form misspecification	Additive form ignores N–D coupling	Fit Skaling and Chinchilla; residual diagnostics (§7)
Early-transient contamination	Early loss is far from asymptotic	Drop first ~10% of checkpoints / first ~10B tokens (§4.4)
Inflated degrees of freedom	Checkpoints on one trajectory are autocorrelated	Block bootstrap over runs (§7.3)
Instability at scale	Overflow, spikes, router collapse dormant at small scale	High-LR stress tests, stepped holdouts (§9.3)
Downstream discreteness	Argmax accuracy discards distractor information	Predict continuous BPB/NLL; COD clustering (§8)
Implementation drift	Different kernels/parallelism compute different updates	Four-step parity check (§9.4)
Data regime change	Exponents are corpus-dependent	Re-fit when data changes (§6.1)

2. Decision objectives and acceptance criteria

ⓘ **Companion reading** ›

2.1 Fix the objective first

Read the detailed companion →

1. **Fixed-recipe extrapolation**: predict how a recipe with prescribed HP scaling rules performs at target scale (no per-point grid search).

2. **Optimal compute allocation**: solve $\arg\min_{N,D:\, C(N,D)=C_0} L$.
3. **Candidate comparison**: does a new architecture/optimizer/mixture beat baseline *at target scale*?
4. **Cross-scale HP prediction**: fit (η, B, λ) as functions of (N, D).

Objectives 2–4 require every fitting point on the fully-tuned frontier; objective 1 only executes the recipe's formulas.

2.2 Evaluation protocol pitfalls

Read the detailed companion →

- Small-scale loss improvements need not become target-scale benchmark gains.
- Hard tasks (math, code) are near chance at small scale while easy tasks saturate at large scale. Use continuous proxies such as **answer bits-per-byte (BPB)** where accuracy is flat (OLMo 3).
- High small-scale discriminability only proves low noise; verify **rank correlation** between small-scale proxy and target-scale metric before selecting on it.
- Multi-sample or isolate high-variance benchmarks.
- Freeze prompt templates, decoding params, scoring scripts; decontaminate training vs eval.
- **Post-training plasticity check.** Some changes are invisible in pretraining loss but break post-training (Qwen3.8-Next: NoPE led to non-terminating generations after post-training; a sparse-read residual branch degraded post-training quality). OLMo 3 runs fast instruction tuning on fully annealed candidate mixtures and checks scores plus generation termination/length.

2.3 Acceptance thresholds

Read the detailed companion →

Four questions: (i) point-prediction error of a fixed recipe at target compute; (ii) predicted margin and ranking confidence between candidates; (iii) loss regret of the chosen (N, D) versus neighbors; (iv) stability and capability floors.

- Fix thresholds **before** looking at holdouts, calibrated to seed noise and minimum meaningful effect. Reference points: adopted modeling changes are typically ~4% relative improvement while seed restarts alone vary ~3.5% (Choshen et al.); Delphi's 0.2–0.5% prediction error is a low-noise regime, not a universal bar.
- For two candidates on the same ladder, estimate confidence intervals on the **paired difference** ΔL (shared noise cancels), not by adding independent error bars.
- Low average loss error does not imply a stable $(N_{\text{opt}}, D_{\text{opt}})$: which grid points enter the fit can visibly move the allocation (Gemstones). Propagate fit uncertainty to the *decision variable*.

---

3. Measurement specification

ⓘ Companion reading

3.1 Parameter count

Read the detailed companion →

- **For fitting** $L(N, D)$: use the trunk count N_{body} (exclude input embedding and output head). Embedding/head scale as $O(Vd)$ vs trunk $O(d^2 n)$, so at small scale they distort exponents. Farseer reports worse extrapolation at 25.1B when embeddings are included (consistent with Kaplan).
- **For counting FLOPs** C: the output head's $d \times V$ matmul runs on every token and *must* be counted (even if tied); the input lookup is memory traffic, logged separately.

- **Convention hazards**: Porian et al. include the head but not input embedding; Chinchilla's 20 TPP counts all embeddings.
- **MoE logging per grid point**: N_{total}, N_{active} (embedding/head separate); E_{total}, E_{active}, sparsity $S = E_{\text{total}}/E_{\text{active}}$; shared-expert count/size; routed expert size (granularity); router and load-balancing config.

3.2 Compute

Read the detailed companion →

For standard MHA with $4d$ FFN and full $L \times L$ attention (forward + backward):

$$C \approx \left(6 + \frac{L}{d}\right) N_{\text{body}} D + 6VdD, \qquad N_{\text{body}} \approx 12d^2 n.$$

[Note] Derivation. Per token per layer, the forward pass computes $QK^\top$ and AV, each $\approx 2Ld$ FLOPs, so $4Ld$ forward; backward is $2\times$ forward, giving $12Ld$. Over n layers: $12Ldn = \frac{L}{d} \cdot 12d^2 n = \frac{L}{d} N_{\text{body}}$. The head costs $2Vd$ forward, $6Vd$ total per token. Causal FlashAttention skipping masked tiles roughly halves the attention term. SwiGLU, GQA, MoE change constants; **always count from the actual architecture and kernels**.

For MoE, matmul FLOPs scale with N_{active} plus router overhead; EP all-to-all adds no FLOPs but costs wall-clock.

3.3 Loss and token conventions (locked across the ladder)

Read the detailed companion →

Dimension	Requirement
Primary loss	Pure next-token CE; log aux losses, z-loss, MTP separately, never fit on them
Eval distribution	Same held-out corpus and domain weights for all candidates; report key domains
Aggregation	Per-token vs per-document averaging; fixed distributed reduction denominator
Packing/masks	Fixed BOS/EOS, document-boundary masks, packing, position IDs, truncation
Token accounting	Distinguish processed tokens, loss-bearing tokens, unique tokens U, epoch exposures
Seeds	Separate init / sampling / shuffle seeds; paired seeds for ablations
Cross-tokenizer	Byte-normalized metrics (BPB); compare on identical raw text

- When the mixture varies, training loss on each run's own data reflects corpus entropy (more code → lower CE) and cannot rank models. Always compare on a fixed validation distribution.
- Evaluate at **fixed fractions** of total steps (e.g., every 2%), not every k steps; otherwise long runs contribute more points and dominate curve fits.

4. Recipe invariants and experimental design

ⓘ **Companion reading** ›

4.1 Variable classification

Read the detailed companion →

Category	Role	Examples
Fixed invariants	Constant over the ladder	Model family, data version, optimizer type, seq length
Prescribed scaling rules	Deterministic function of N or D	LR (power law or μP), BSZ ($\propto D^{0.4}$ prior), warmup *fraction*
Independent variables	Swept or solved	N, D, tuned optimal LR and BSZ

Classic pitfall: a fixed warmup *step count* eats a larger fraction of short runs and biases the compute-optimal exponent (Porian et al.). Use a fixed fraction of tokens (Delphi: 10%). Changing architecture, optimizer, data version, precision, or tokenizer can shift exponents.

Identifiability. If all runs lie on one TPP ray $D = cN$, then $\log D = \log N + \log c$, the design matrix in $(\log N, \log D)$ is rank-deficient, and α, β are not separately estimable. Inspect the parameter covariance or hold out a size tier.

4.2 Architectural and training invariants

Read the detailed companion →

Identical across sizes: backbone topology, normalization type/placement (e.g., pre-RMSNorm), positional scheme and base (RoPE), activation and FFN ratio (SwiGLU), embedding tying, attention type/grouping (GQA), a single width–depth progression rule, MoE routing/balancing/shared-expert ratio. Training: seq length (typ. 4096), frozen data snapshot and mixture, optimizer (AdamW), same validation set, eval at fixed progress fractions, same tokenizer/init/parameterization/loss reduction.

Proxy models should mirror the target's structural ratios (GQA group ratio, `head_dim`, width/depth progression) so memory-bandwidth behavior and representational dynamics transfer.

4.3 Width–depth

Read the detailed companion →

At fixed N_{body}, aspect ratio changes the attention FLOP share L/d. Kaplan found pretraining loss flat across a broad range of aspect ratios, but Gemstones and GLM-4.5 find it affects downstream and reasoning performance. Enforce one width–depth rule; add targeted controls when depth or head dim is studied.

4.4 MoE structural rules

Read the detailed companion →

- **Granularity**: expert size is its own scaling dimension; "expert = dense FFN size" is suboptimal at nearly all budgets; finer experts widen MoE's edge with scale (Krajewski et al.).
- **Shared experts**: fix count and capacity ratio.
- **Load balancing**: fix aux-loss coefficient or bias update rate. DeepSeek-V3 uses bias update speed 0.001 for the first 14.3T tokens, 0 for the final 500B. Changing balancing strength is a recipe change (trades loss vs EP throughput).
- **Capacity/token dropping**: consistent train/inference capacity; log drop rate (DeepSeek-V3 drops none).
- **Sparsity**: at fixed training FLOPs, higher sparsity (larger N_{total}) lowers loss; at fixed N_{total}, loss is U-shaped in sparsity with optimum moving up with compute (Abnar et al.). Downstream tracks loss except reading comprehension (CoQA, SQuAD) where denser models keep an edge.

4.5 Vocabulary and precision

Read the detailed companion →

- Optimal vocab grows as a power of compute (Tao et al., 33M–3B); most open models are under-vocabularied. Changing V changes head FLOPs and compression, so compare in BPB.
- Lower training precision reduces effective parameter capacity; higher TPP amplifies post-training-quantization degradation (Kumar et al., up to 1.7B / 26B tokens). Run the ladder in the target precision.

---

5. Grid design and budget

> ⓘ **Companion reading**

5.1 Grid shape

Read the detailed companion →

```
D multiplier →   0.5×   1×   2×   4×
N  130M           ●     ●     ●     ●
   520M           ●     ●     ●     ●
   2.3B           ●     ●     ●     ●
   8B             ○     ○     ○     ○   ← holdout, never fitted
```

Choose full Cartesian, IsoFLOP slices, or L-shape based on the objective; reserve validation points and contingency budget.

5.2 Sizes, span, extrapolation multiplier

Read the detailed companion →

Dimension	Guideline
Number of sizes	≥ 3–4 fitting sizes plus holdouts
Span	Wide; power-law linearity holds over a $34\times$ span for several families (Choshen et al.)
Adjacent ratio	Geometric, typically $2\times$–$4\times$

Extrapolation multiplier = target FLOPs / largest fitting-point FLOPs. Delphi: fit 3×10^{18}–3×10^{20}, validate at $3\times$–$333\times$. Llama 3: fit 6×10^{18}–10^{22} (40M–16B), extrapolate ~$3{,}800\times$ to 3.8×10^{25}. Large multipliers make second-order misspecification and late instability dominate; insert a dress rehearsal (§9.6).

5.3 Token-horizon tiers

Read the detailed companion →

- **Chinchilla IsoFLOP**: for each fixed C, train several N, fit loss vs $\log N$ (approximately quadratic), read off the minimum; fit $N_{\text{opt}}(C)$, $D_{\text{opt}}(C)$. Llama 3 used this at 3.8×10^{25} to estimate ~402B, chose 405B.
- **Near compute-optimal**: $0.5\times$–$4\times$ Chinchilla; Fantastic Optimizers uses $1, 2, 4, 8\times$.
- **Over-training**: power laws hold up to $32\times$ Chinchilla (Gadre et al., 104 models). Up to 10,000 TPP, loss still improves but coefficients fitted at low TPP *overestimate* marginal returns of extra tokens (Sardana et al.). Include high-TPP fitting and holdout points if targeting high TPP.
- **Inference-amortized**: including lifetime inference cost shifts the optimum to smaller N, larger D (Llama 3 8B/70B heavily over-trained at >15T; 405B near compute-optimal).

- **Data-constrained**: annotate epochs; add a repetition term (§6.3).

5.4 Intermediate checkpoints, seeds, shared trunks

Read the detailed companion →

- **Early transients**: discard the first ~10% of checkpoints or first ~10B tokens; this substantially reduces extrapolation error (Choshen et al.). Fix the truncation rule *before* fitting, or tune it on a separate validation size tier (Lourie et al.), never on the final holdout.
- **Autocorrelation**: checkpoints on one trajectory carry far fewer effective degrees of freedom. WSD branches forked off a shared stable trunk share prefix noise: don't double-count trunk FLOPs; treat branches as one correlated cluster in bootstrap.

5.5 Budget staging

Read the detailed companion →

1. Pilots: measure seed variance, throughput, rough effect sizes.
2. Split main budget across HP tuning, grid fitting, holdouts; keep **15–20% reserve**.
3. Spend reserve by diagnosis: more seeds if seed noise dominates; more points where candidate forms (Chinchilla vs Skaling) disagree; longer horizons if candidate curves cross late.

5.6 MoE axes (factor instead of a full Cartesian product)

Read the detailed companion →

1. **Scale ladder**: fix S and granularity; scale N_{active}, D.
2. **Sparsity ladder**: fix $N_{\text{active}}, D, E_{\text{active}}$; sweep E_{total} (Kimi K2: sparsity 8→48 steadily reduces compute to reach a target loss).
3. **Granularity ladder**: fix $N_{\text{active}}, N_{\text{total}}$; trade expert size against E_{active}.

Don't pool axes unless the equation models those variables; include iso-N_{active} or iso-FLOP dense baselines.

---

6. Hyperparameters

ⓘ Companion reading

6.1 The sensitivity asymmetry

Read the detailed companion →

Small models are sharply sensitive to HPs; large models have a wide, flat basin (Lourie et al.; Step Law). Example: on a 156B-A7B model, LR $\times/\div\sqrt{2}$ and BSZ $+25\%$ changed final training loss by at most 7×10^{-4} (Qwen3.8-Next). Strategy: dense grids at small scale with the optimum strictly **interior** and local perturbation deltas below seed noise; only narrow local checks at large scale.

6.2 Parameterization and optimizer transfer

Read the detailed companion →

- **Width**: μ-Transfer rescales per-layer init and LR so optimal LR transfers across width; weight-decay variants in Tensor Programs V App. G.1.2 and Power Lines. μP does **not** handle batch size or horizon D; combine with empirical (N, D)

laws.

- **Depth**: $1/\sqrt{\text{depth}}$ residual scaling gives depth transfer in ResNets/ViTs (Bordelon et al.), but Tensor Programs VI shows exact infinite-depth transfer fails when a residual block has multiple nonlinear layers, as in Transformers. Recalibrate empirically when depth changes.
- **Optimizer switch**: Muon with weight decay and update RMS matched to AdamW's typical 0.2–0.4 range (set to 0.2) lets AdamW LR/WD transfer (Liu et al.). But changing architecture and switching to Muon simultaneously shifted both optimal LR and BSZ (Qwen3.8-Next). Re-verify at small scale.

6.3 LR and batch-size laws

Read the detailed companion →

Single fixed TPP (DeepSeek LLM):

$$\eta_{\text{opt}} = a\,C^{-\alpha_\eta}, \qquad B_{\text{opt}} = b\,C^{\beta_B}.$$

Across TPPs, decouple N and D:

Approach	η_{opt}	B_{opt}	Notes
Step Law (Li et al.)	$c\,N^{-\alpha}D^{+\beta}$	$0.58\,D^{0.571}$	3,700+ models; B_{opt} depends on D, ~invariant to N
Power Lines (Bergsma et al.)	via timescale τ	$\propto D^{0.4}$	Unifies (η, B, λ); bounded by max stable LR and B_{crit}
Token Horizons (Bjorck et al.)	$c\,N^{-\alpha}D^{-\beta}$	fixed	Longer horizon → lower LR at fixed B

Sign flip on D. Token Horizons holds B fixed, so more steps require a smaller step size. Step Law co-scales $B_{\text{opt}} \propto D^{0.571}$; the gradient-noise reduction from larger B outweighs the longer horizon, so η_{opt} rises slightly with D. Both are consistent; they answer different conditional questions.

Consensus: $B_{\text{opt}} \propto D^{0.4\text{–}0.57}$, nearly independent of N.

Critical batch size. Power Lines: $B_{\text{crit}} \propto D_{\text{min}}^{0.5}$, where D_{min} is tokens to reach a target loss at small batch.

[Note] The hyperbola. The steps–tokens trade-off to reach a fixed loss is (McCandlish-style)

$$\left(\frac{S}{S_{\text{min}}} - 1\right)\left(\frac{D}{D_{\text{min}}} - 1\right) = 1, \qquad B = D/S.$$

At $B_{\text{crit}} := D_{\text{min}}/S_{\text{min}}$, the symmetric point gives $S = 2S_{\text{min}}$, $D = 2D_{\text{min}}$: the knee where further batch increases buy few step reductions at steep token cost.

Forgiving basin. Batch sizes within ~5% compute overhead span a $4\times$ range, i.e., $[B_{\text{opt}}/2, 2B_{\text{opt}}]$ under a log-symmetric model (Schaipp, 2026). Use this slack to fit hardware multiples.

Practical rules. Measure B in tokens; if the production run ramps batch size (Llama 3: 4M→16M tokens in stages; Nemotron-4 similar) but the ladder doesn't, test the ramp's effect.

6.4 Weight decay and the EMA timescale

Read the detailed companion →

Several recipes fix WD ($\lambda = 0.1$ in Kimi K2; OLMo 3 fixed WD with no decay on embeddings; Qwen3 doesn't scale WD). But the loss-optimal WD decreases monotonically with TPP, while somewhat higher WD preserves effective rank and post-training plasticity (Han et al.).

Under AdamW, the weights are an EMA of updates with time constant $1/(\eta\lambda)$ iterations. Expressed as a fraction of training ($S = D/B$ steps):

$$\tau = \frac{1/(\eta\lambda)}{D/B} = \frac{B}{\eta\lambda D}.$$

Power Lines finds the optimum keeps τ_{opt} constant, so at fixed (μP) η, set $\lambda \propto B/D$ — provided η is below the maximum stable LR.

6.5 LR schedules

Read the detailed companion →

- **Cosine**: warmup then cosine to 0 or ~10% of peak; good when D is fixed in advance.
- **WSD**: constant LR then a sharp final decay. River-valley picture: the stable phase moves along the valley floor; the decay settles into the walls. Main ladder advantage: **fork multiple decay branches from one trunk** to cover several D tiers.
- Decay fraction 10–20% of steps is a strong baseline (Tissue et al.).
- Caveat: once each schedule had its own tuned HP law, cosine beat WSD at Kimi K3 and became the default — cosine and WSD optima sit at very different (η, B).

6.6 MoE HPs

Read the detailed companion →

Fit $(\eta_{\text{opt}}, B_{\text{opt}})$ on the MoE family directly (routing changes gradient noise). Lock router knobs (balance coefficient or bias rate, FP32 router logits, capacity factor). Check expert load and drop rate at intermediate scale.

6.7 Search procedure

Read the detailed companion →

Sensitivity hierarchy: peak LR, BSZ ≫ schedule, WD ≫ β_2, ϵ.

1. Full 2-D (η, B) grid at ~130M; optimum strictly interior.
2. Narrower grid at ~500M to anchor the slope.
3. At large scale, check $\eta \in [\eta_{\text{pred}}/\sqrt{2}, \sqrt{2}\,\eta_{\text{pred}}]$, $B \in [B_{\text{opt}}/2, 2B_{\text{opt}}]$; expand only if a boundary point wins.

Use N-invariance of $B_{\text{opt}}(D)$ and τ-calibrated λ to collapse the 3-D search.

Winner's curse. The minimum over noisy trials is biased downward. Re-run the winner with a fresh seed and fit on that. Never prune before annealing: rankings flip during LR decay.

6.8 Fair recipe comparisons

Read the detailed companion →

- Match tuning effort (many "better optimizers" beat an untuned AdamW; Fantastic Optimizers).
- Compare at the annealed endpoint only.

- Track gains across ≥ 3 sizes: matrix-optimizer token-efficiency over AdamW at $8\times$ Chinchilla fell from $1.4\times$ (0.1B) to $1.1\times$ (1.2B); token savings ≠ wall-clock savings.

6.9 Wrap-up operations

Read the detailed companion →

Include in the ladder anything the production run does at the end: cooldown mixture shift to high-quality/reasoning data (Nemotron-4, OLMo 2); checkpoint averaging (Model Soups, LAWA). ByteDance Seed's PMA: merging checkpoints during the WSD stable phase approximates fully annealed downstream performance at 1.3B and 13B — a cheap proxy for annealing.

---

7. Data ladder

ⓘ Companion reading

7.1 Data quality

Read the detailed companion →

- Higher-quality data shifts compute-optimal allocation toward **larger** N; exponents are **corpus-dependent** (DeepSeek LLM). Kimi K3 rebuilt its laws from scratch when changing architecture, corpus, and recipe together.
- Cheap screening of a new source:
  - **Micro-annealing** (OLMo 2): fork a mid/late checkpoint, anneal briefly on baseline + candidate, measure downstream lift. Captures late-stage marginal value, not full-run ranking.
  - **Small-model domain ablations** (Kimi K3).
  - **Targeted synthetic injection**: a legal synthetic corpus raised Nemotron 3 Nano's LegalBench proxy from 64.6 to 74.7.

7.2 Mixture optimization pipeline (Olmix + OLMo 3 + Marin)

Read the detailed companion →

Fix N, D, optimizer; vary $\mathbf{w} = (w_1, \ldots, w_m), \sum_j w_j = 1$.

1. **Bucket and inventory**: by source/domain/tags; re-count unique tokens N_j whenever filters change. For staged runs, optimize each stage separately (Nemotron 3 Super: diversity for first 80% of 25T, quality for last 20%).
2. **Baselines**: natural token-proportional and UniMax; poorly constrained learned mixtures often lose to both.
3. **Proxies**:
   - Size: at $5\times$ Chinchilla, proxies $$15M params give Spearman > 0.89 with a 1B target; 1M drops to 0.73 (Olmix). OLMo 3 uses 30M proxies for 3B tokens.
   - **Proportional pool downscaling** (Marin): shrink each bucket's available tokens in proportion to proxy active params so the proxy sees the *same repetition rate* as the target (791 tokens per active param in their ladder).
   - Samples: $\geq 3(m+1)$ Dirichlet mixtures around the natural prior for a log-linear surface. DeMix: one model per domain + weighted model merging gives higher rank consistency than retrained proxies.
4. **Per-task regression**: fit a log-linear model per eval task, then aggregate — 0.983 correlation on held-out mixtures vs 0.866 for fitting the aggregate directly. The response is non-monotone: raising a small bucket's weight first helps, then plateaus, then hurts once repetition bites. Because of the simplex constraint, single-bucket marginal correlations are confounded and can flip sign between pretraining and cooldown.

5. **Constrained solve**: per-bucket epoch caps

$$w_j \leq \frac{k\,N_j}{R}$$

(R target tokens, k max epochs), plus a light KL regularizer ($\lambda = 0.05$) toward the natural distribution.
6. **Cross-scale confirmation** against proportional and UniMax at intermediate scale.
7. **Incremental reuse** (Olmix): freeze relative weights of unchanged buckets, re-run proxies only on the changed subspace. Over 5 updates to 64 domains (1B, 100B tokens): +11.6% over natural sampling, 95% of full-recompute gains, 74% fewer proxy runs; used for three OLMo 3 mixture iterations.

7.3 Repetition

Read the detailed companion →

Prerequisite: global cross-source dedup, so "1 epoch" is meaningful. Marin found large hidden overlap between Nemotron-CC and its synthetic rewrites.

Muennighoff et al. effective data:

$$D' = U_D + U_D\,R_D^*\left(1 - e^{-R_D/R_D^*}\right), \qquad \lim_{R_D\to\infty} D' = U_D\,(1 + R_D^*),$$

with U_D unique tokens and R_D repetitions *beyond* the first epoch.

[Note] The post states $R_D^* \approx 4$. As far as I know, the fitted value in Muennighoff et al. is $R_D^* \approx 15.4$ (and $R_N^* \approx 5.3$ for the parameter-side term); "~4 epochs" is the regime where repetition is nearly as good as fresh data. With $R^* \approx 15$, $e^{-40/15} \approx 0.07$, consistent with returns saturating near 40 epochs in their Fig. 1. Also, R^* is an e-folding scale; the half-life is $R^* \ln 2$. Check the paper before using a number.

Further findings: - Overfitting grows with N and $1/U$ and repetition; optimal repetition grows ~logarithmically with dataset size in linear regression (Yan et al.). With fixed U, scaling N aggressively hurts, pushing the optimum to smaller N. - Higher quality does not prevent multi-epoch overfitting; re-tuned **dropout** helps (Xue et al.). - A small specialist corpus diluted in a large general mix tolerates **15–20 repetitions** with positive returns (Sedova et al., 2,000+ runs). - Quality-tiered epoch caps: OLMo 3 upsamples only top buckets up to 7 epochs, beating aggressive threshold filtering. - **Synthetic rewriting** extends effective unique data. Kimi K2 SimpleQA at an early checkpoint: raw ×10 epochs = 23.76; one rewrite ×10 epochs = 27.39; 10 distinct rewrites ×1 epoch = 28.94 (production cap: 2 rewrites per corpus). Count rewritten tokens separately from U.

---

8. Fitting the loss law

ⓘ Companion reading

8.0 Pre-fit checklist (Porian et al.'s three confounders)

Read the detailed companion →

1. C includes head FLOPs; N_{body} used for fitting.
2. Warmup scales with D.
3. (Objectives 2–4) all points on the fully-tuned frontier.

8.1 Forms

Read the detailed companion →

Default candidates: Chinchilla and Skaling (§1.2). Select on a held-out validation slice.

8.2 Whole-curve law (LR annealing)

Read the detailed companion →

Tissue et al.:

$$\hat{L}(s) = L_0 + A\, S_1(s)^{-\alpha} - C\, S_2(s), \qquad S_1(s) = \sum_{i \le s} \eta_i,$$

where S_2 accumulates LR *decreases* passed through an exponential-forgetting momentum kernel (annealing benefit). With the schedule as an input, one fit predicts cosine, WSD branches, and re-warmup curves using many intermediate points.

8.3 Fitting protocol

Read the detailed companion →

Naive `curve_fit` is fragile; small changes in init, weighting, or outlier handling move $(N_{\text{opt}}, D_{\text{opt}})$ by multiples (Besiroglu et al.; (Mis)Fitting).

- **Target and loss**: decide L vs $\log L$ residuals and whether E is free or prior-bounded; Huber loss; weight trajectories inversely by checkpoint count.
- **Multi-start global optimization**: optimize log-parameters with box bounds, hundreds of random inits (multi-start L-BFGS-B), because coefficients and exponents trade off along curved valleys.
- **Block bootstrap**: resample independent runs (or trunk-branch clusters), not checkpoints; **re-solve the decision quantity** in each replicate.

8.4 Diagnostics

Read the detailed companion →

- Plot signed residuals against $\log N$, $\log D$, TPP; saddle or monotone tail curvature = misspecification.
- Make sure autocorrelated checkpoints are down-weighted (Delphi bootstraps only over IsoFLOP parabola minima).
- Compare holdout error to multi-seed SD: near seed SD → noise floor; much larger → revisit truncation or add points along the failing axis.

8.5 MoE

Read the detailed companion →

Scale ladder: substitute N_{active} for N. Sparsity/granularity sweeps: include them explicitly in the form (Krajewski; Ludziejewski); never pool different sparsities into one (N_{active}, D) fit. Compare to a dense fit in equivalent-compute coordinates.

---

9. Downstream prediction

ⓘ **Companion reading** ›

9.1 Three routes

Read the detailed companion →

Route	Chain	Examples	Trade-off
Loss → Acc	$(N, D) \to L \to$ monotone map	Gadre et al. (power law in perplexity); Delphi (sigmoid)	Simple; vulnerable to pretraining/task distribution shift
Task NLL → Acc	$(N, D) \to$ correct-answer NLL $\to$ Acc	OLMo task ladder; Llama 3	Avoids domain mismatch; stage 2 noise varies by benchmark
Difficulty buckets	Cluster items by small-model pass rates; fit per cluster	COD; GPT-4 HumanEval	Handles near-zero small-model accuracy; needs multi-sampling and stable clusters

Llama 3: stage 1 fits normalized correct-answer NLL linearly in $\log$ FLOPs using runs $\leq 10^{22}$; stage 2 fits a sigmoid NLL→accuracy using ladder models plus Llama 2 checkpoints. Predictions for 405B (e.g., ARC-Challenge) were tight, slightly conservative.

9.2 COD (Clustering-on-Difficulty)

Read the detailed companion →

1. Multi-sample each item across small models; per-model pass-rate vector = difficulty signature; cluster.
2. Per cluster, fit

$$\mathbb{E}[\mathrm{Acc}(C)] = g + (1 - g)\, e^{-aC^{-b}-c},$$

   with guessing floor g.
3. Drop clusters stuck at g or already saturated; extrapolate the rest; size-weighted average.
4. Monotone calibration from subset score to full benchmark.

Reported: 1.55 pp mean absolute error at 70B over 8 benchmarks. Caveats: small benchmarks → noisy clusters; architecture/mixture changes reshuffle difficulty; proxies must match the two-stage data schedule and TPP; dense→MoE (32B active) transfer gave MAE 3.11 pp, max 8.11 pp; no theory for open-ended CoT.

9.3 Limits

Read the detailed companion →

Multiple-choice accuracy depends on the probability on specific distractors, not only on the correct answer; each step from CE to argmax discards information (Schaeffer et al.). Validate task predictions on holdouts; when they fail, base decisions on continuous BPB or loss.

---

10. Validation and launch

ⓘ **Companion reading** ›

10.1 Holdouts

Read the detailed companion →

Place holdouts *along the extrapolation direction*, in multiple steps. Delphi's stepped holdouts caught both the drift at $33\times$ and the divergence at $333\times$. **A holdout is unbiased only once**: if it informs any change (recipe, truncation, form), it becomes development data.

10.2 Combined validation

Read the detailed companion →

Gains don't add: changes share gain pools or interact (aggressive optimizer → faster memorization of repeated buckets; new mixture → shifted LR/WD optima). Validate the combined recipe end-to-end at target TPP and repetition.

10.3 Stability stress tests

Read the detailed companion →

Run medium-scale models at $2\times$ and $4\times$ the predicted LR to compare divergence thresholds (Qwen3.8-Next). Not sufficient for multi-T-token late failures: Nemotron 3 Ultra had two late divergences — one fixed by FP32 output-layer gradients, one with no identified cause, mitigated by annealing early. Monitor per-layer grad norms, activation maxima, spike counts; for MoE, max load imbalance, drop rate, router entropy drift.

10.4 Implementation parity (research stack vs production stack)

Read the detailed companion →

1. **Single-step parity**: same checkpoint and batch → logits, loss, per-layer grad norms, and one-step updates match within reduction tolerance.
2. **Short-trajectory parity**: fixed seeds for hundreds–thousands of steps; no systematic drift.
3. **Parallelism/reduction audit**: effective global batch, loss/gradient denominators (variable lengths, padding), clipping order, accumulator precision.
4. **Stateful resume**: optimizer moments, scheduler step, RNG states, dataloader cursors restore exactly (no skipped/repeated windows).

References: DeepSeek-V3 FP8 precision rules; Llama 3 deterministic dataloading and recovery.

10.5 Realized efficiency

Read the detailed companion →

Measure MFU, tokens/s, memory headroom, effective daily progress after eval/checkpoint/failure overhead; EP all-to-all latency on real topology. On the serving side, measure cost under real prefill/decode distributions, concurrency, TTFT/TPOT SLAs, and quantization; plug into the Sardana et al. lifecycle model.

10.6 Dress rehearsal

Read the detailed companion →

If target compute exceeds the largest holdout by more than ~$100\times$: run an intermediate model on the exact production stack, precision, and combined mixture; freeze the predicted curve and band *before* launch; gate the flagship on staying in band with clean stability metrics.

---

11. Staged and long-context ladders

ⓘ **Companion reading** ›

- Stages (pretrain → mid-train/anneal → long-context) each have different data, sequence length, schedule, and exponents. Build a ladder per stage where decisions are quantitative. Stage ladders depend on the parent checkpoint (same loss can mean different domain exposure, optimizer state, LR history); test optimizer reset, re-warmup, and carried-over repetition across several parents, and validate the chained pipeline.
- Long-context: control short/long mixture, key-information position, document masks; track short-context regression. Needle-in-a-haystack ≠ multi-hop tracing or aggregation (RULER).

---

12. Operations

> ⓘ Companion reading

- **Provenance per run**: commit, full config, data manifest/mixture hash, tokenizer version, eval script version, all seeds, parent checkpoint and shared prefix steps, hardware topology and precision. Status codes for designed early stops, divergences, crashes, bug-invalidated runs; keep diverged runs (they mark the stability boundary).
- **During flagship training**: the predicted curve is a live instrument. Isolated excursions are usually eval noise; sustained divergence → check eval scripts/token counters/config, then dataloader/reductions/kernel parity, and only then blame the extrapolation.
- **Maintenance**: golden regression configs re-run on framework/kernel/hardware changes; versioned coefficients with data/model/script; incremental recalibration (2–3 small points first, full rebuild only if deviation exceeds seed noise).

Appendix A. Public ladders and reference configurations

> ⓘ Companion reading

A.1 Scales

Read the detailed companion →

Source	Params	Tokens / compute
Kaplan et al.	up to 1.5B non-embedding	22M–23B tokens
Chinchilla	70M–16B (400+ models)	5B–500B tokens
Step Law	3,700+ models	~100T tokens total
Gadre et al.	11M–6.9B (104 models)	up to $32\times$ Chinchilla
Fantastic Optimizers	0.1B–1.2B (4 sizes)	1–$8\times$ Chinchilla
Delphi	max holdout 25B	fit 3×10^{18}–3×10^{20}; holdouts to 10^{23}
Llama 3	40M–16B	fit 6×10^{18}–10^{22}; target 3.8×10^{25}

Production references: Nemotron-4 15B/340B at 8T/9T tokens; OLMo 1 at 2–2.46T, OLMo 2 up to 5T+; Llama 3 405B at 15.6T.

A.2 Fantastic Optimizers ladder (dense, Llama-2-style)

Read the detailed companion →

Depth fixed at 32 layers, MHA, seq 4096; size varies through width only. Data: DCLM-baseline + StarCoder V2 + ProofPile 2. Every (size, horizon) point independently tuned.

Size	d	FFN	Heads	Horizons
130M	512	2048	8	1–$8\times$ Chinchilla
300M	768	3072	12	same
520M	1024	4096	16	same
1.2B	1536	6144	24	same

Example AdamW optimum at one tier: peak LR 8×10^{-3}, WD 0.1, warmup 2000 steps, BSZ 128 seqs (~0.5M tokens); at 520M/$1\times$, WD 0.2 and BSZ 256.

A.3 Delphi ladder (Marin)

Read the detailed companion →

Qwen3-style decoder (MLP ratio 4, seq 4096), later extended to MoE (535B-A23B). AdamH, WSD (10% warmup, 20% linear decay to 0), FP32 master / BF16 compute, FSDP. Data: Nemotron-CC + StarCoderData + ProofPile 2. Seven IsoFLOP slices from 3×10^{18} to 3×10^{20}; power law through slice minima; HPs from prescribed formulas; holdouts 10^{21}–10^{23} (to 25B params, $3\times$–$333\times$). The first attempt (Cautious AdamC) is the one that drifted and diverged.

A.4 Which template

Read the detailed companion →

- HP laws and fair algorithm comparisons → Fantastic Optimizers-style $(N \times \mathrm{TPP})$ Cartesian grid, all points tuned.
- Compute allocation and loss forecasting → Delphi-style IsoFLOP + formula HPs + stepped holdouts.
- MoE → start from a dense template, factor along the three MoE axes, match GQA and aspect-ratio progression to target.

Appendix B. Numbers worth remembering

ⓘ **Companion reading** ›

Quantity	Value	Source
Chinchilla exponents	$a \approx b \approx 0.5$	Hoffmann et al.
Skaling coupling	$k \approx 0.31$–0.45	Videau et al.
B_{opt}	$\propto D^{0.4}$ to $0.58\, D^{0.571}$, ~indep. of N	Power Lines; Step Law
B near-optimal window	$[B_{\text{opt}}/2,\ 2B_{\text{opt}}]$ at ≤5% compute	Schaipp
B_{crit}	$\propto D_{\min}^{0.5}$	Power Lines
WD timescale	$\tau = B/(\eta\lambda D)$ held constant	Power Lines
Warmup	fixed *fraction* (Delphi 10%)	Porian; Delphi
Decay fraction	10–20% of steps	Tissue et al.
Early truncation	first ~10% ckpts or ~10B tokens	Choshen et al.
Reserve budget	15–20%	post
Seed noise vs typical gain	~3.5% vs ~4%	Choshen et al.
Power law over-training validity	to $32\times$ Chinchilla	Gadre et al.

Quantity	Value	Source
Mixture proxy size	≥15M params (ρ>0.89 to 1B)	Olmix
Mixture samples	$\geq 3(m+1)$	post
Diluted specialist repetition	15–20× still positive	Sedova et al.
LR stress test	$2\times, 4\times$ predicted	Qwen3.8-Next
Large-scale HP verification	$\eta \times / \div \sqrt{2}, B \times / \div 2$	post

Appendix C. Condensed checklist

ⓘ **Companion reading**

Design. Regime + objective + eval protocol; metrics with signal at ladder scale; post-training checks if shipping a post-trained model; thresholds pre-registered in compute-multiplier units; variable roles fixed; warmup as fraction; ≥3–4 sizes + stepped holdouts; TPP tiers bracket target; budget with 15–20% reserve; MoE axes factored; data frozen and globally deduped; mixtures vs natural and UniMax with proportional pool downscaling.

Measurement. N_{body} for fits, full FLOPs for C; locked CE/eval set/token accounting; eval at progress fractions; target precision; BPB across tokenizers.

HPs. Interior optima at small scale; re-verify after optimizer/depth changes; B in tokens with $B_{\text{opt}}(D)$ prior; WD via τ; fresh-seed re-run of winners; no pre-anneal pruning.

Fit. Pre-fit confounders cleared; Chinchilla vs Skaling on validation slice; repetition term if multi-epoch; truncation rule frozen; multi-start robust fit; block bootstrap to the decision; residual diagnostics.

Launch. One-shot holdouts; combined recipe validated; $2\times$ / $4\times$ LR stress; parity + resume checks; measured throughput and serving constraints; dress rehearsal for big gaps; full provenance archived.